Базы данных в Python

Практическое руководство по SQL и NoSQL

2025

Содержание

Введение: что такое база данных 1Зачем нужны базы данных 2Проблемы хранения данных без БД 2Преимущества использования баз данных 2SQL и NoSQL: два подхода к хранению данных 3Реляционные базы данных (SQL) 3Нереляционные базы данных (NoSQL) 4Сравнение SQL и NoSQL 5SQLite: работа с базой данных в Python 6Подключение и создание таблиц 6Основные операции CRUD 7Выборка данных с фильтрацией 8PostgreSQL: продвинутая работа 9Установка psycopg2 9Пример подключения и запросов 9MongoDB: работа с NoSQL в Python 10Установка PyMongo 10Основные операции 11ORM в Python: SQLAlchemy 12Что такое ORM 12Пример использования SQLAlchemy 12Практические рекомендации 14Когда выбирать SQL 14Когда выбирать NoSQL 14Лучшие практики 1

Примечание: для корректного отображения номеров страниц щёлкните правой кнопкой мыши по оглавлению и выберите «Обновить поле».

Введение: что такое база данных

База данных — это организованная совокупность структурированной информации, хранящейся в электронном виде. Базы данных создаются для того, чтобы систематизировать большие объёмы данных, обеспечивать быстрый доступ к ним и поддерживать их целостность. В современной цифровом мире базы данных окружают нас повсюду: от телефонной книжки в смартфоне до гигантских систем хранения банковских транзакций.

Главное отличие базы данных от обычного файла заключается в том, что база данных управляется специальной программой — системой управления базами данных (СУБД, или DBMS — Database Management System). СУБД отвечает за запись, чтение, обновление и удаление данных, а также за обеспечение безопасности, поддержание связей между таблицами и обработку одновременных запросов от множества пользователей.

Классические примеры использования баз данных: электронная адресная книга, где каждый контакт имеет имя, номер телефона и адрес; интернет-магазин, хранящий каталог товаров, заказы и информацию о покупателях; банковская система, оперирующая миллионами транзакций в день; социальные сети, сохраняющие профили пользователей, сообщения и медиафайлы. Во всех этих случаях данные должны быть структурированными, легко доступными и надёжно защищёнными.

Существует два основных подхода к организации баз данных: реляционный (SQL) и нереляционный (NoSQL). Первый основан на строгих табличных структурах и математическом аппарате реляционной алгебры, второй — на гибких моделях данных, которые могут адаптироваться к изменяющимся требованиям. Python, благодаря богатой экосистеме библиотек, позволяет работать с обоими типами баз данных легко и эффективно.

Зачем нужны базы данных

Проблемы хранения данных без БД

На ранних этапах разработки программисты часто хранят данные в обычных текстовых файлах, форматах CSV или JSON. Хотя такой подход кажется простым, он порождает множество серьёзных проблем. Во-первых, отсутствует возможность одновременного доступа нескольких пользователей: если два процесса попытаются одновременно записать данные в один файл, это приведёт к повреждению информации. Во-вторых, текстовые файлы не поддерживают эффективный поиск и фильтрацию — для нахождения нужной записи приходится загружать весь файл в память и перебирать его строку за строкой.

Кроме того, при работе с файлами возникает проблема дублирования данных. Одну и ту же информацию (например, адрес клиента) приходится записывать в нескольких местах, что затрудняет обновление и ведёт к противоречиям. Файловое хранение не поддерживает целостностные ограничения — нет механизма, гарантирующего, что, например, возраст клиента является положительным числом или что поле электронной почты заполнено корректно.

Масштабирование файлового хранения также представляет серьёзную сложность. Когда объём данных превышает несколько мегабайт, производительность drastically падает. Файлы невозможно легко распределить между несколькими серверами, и создание резервных копий требует ручного копирования всей файловой системы, что подвержено ошибкам и потере данных.

Преимущества использования баз данных

Базы данных решают все описанные выше проблемы и предоставляют широкий набор преимуществ, делающих их незаменимыми для серьёзных приложений:

- Высокая скорость: индексы и оптимизированные алгоритмы обеспечивают мгновенный доступ даже к миллионам записей.
- Параллельный доступ: СУБД корректно обрабатывает одновременные запросы от множества пользователей и процессов.
- Свойства ACID: атомарность, согласованность, изолированность и долговечность гарантируют надёжность транзакций.
- Целостность данных: ограничения, первичные и внешние ключи предотвращают появление некорректных или противоречивых данных.
- Масштабируемость: данные можно распределить между несколькими серверами (шардирование, репликация).
- Безопасность: система авторизации, шифрование и разграничение прав доступа защищают конфиденциальную информацию.

SQL и NoSQL: два подхода к хранению данных

Реляционные базы данных (SQL)

Реляционные базы данных основаны на реляционной модели, предложенной Эдгаром Коддом в 1970 году. Данные организованы в виде таблиц (отношений), каждая из которых состоит из строк (записей) и столбцов (полей). Каждая строка однозначно идентифицируется первичным ключом — уникальным идентификатором записи. Связи между таблицами устанавливаются с помощью внешних ключей, которые ссылаются на первичные ключи других таблиц.

Язык SQL (Structured Query Language) позволяет выполнять разнообразные операции: создавать и изменять структуру таблиц (DDL — Data Definition Language), добавлять, обновлять и удалять данные (DML — Data Manipulation Language), управлять правами доступа (DCL — Data Control Language). Оператор JOIN объединяет данные из нескольких таблиц, что делает реляционные базы данных особенно мощными для сложных аналитических запросов.

Нормализация — важнейший принцип проектирования реляционных баз данных — заключается в устранении избыточности данных и разделении информации по тематическим таблицам. Существует несколько нормальных форм (1НФ, 2НФ, 3НФ и т.д.), каждая из которых устраняет определённый тип аномалии. К популярным реляционным СУБД относятся SQLite (встроенная, без сервера), PostgreSQL (мощная открытая система), MySQL (самая распространённая в веб-разработке) и Microsoft SQL Server.

Реляционные базы данных идеально подходят для задач, где важна строгая структура, целостность данных и поддержка сложных транзакций: финансовые системы, учётные системы, CRM, ERP и другие корпоративные приложения.

Нереляционные базы данных (NoSQL)

NoSQL-базы данных появились как ответ на растущие потребности современных веб-приложений в обработке огромных объёмов неструктурированных данных, высокую доступность и горизонтальное масштабирование. В отличие от реляционных баз, NoSQL не требует заранее определённой схемы и позволяет хранить данные в различных форматах.

Существует четыре основных типа NoSQL-баз данных. Документоориентированные хранилища (MongoDB, CouchDB) хранят данные в виде документов, обычно в формате JSON или BSON. Каждый документ может иметь собственную структуру, что обеспечивает гибкость при изменении требований. Ключ-значение хранилища (Redis, Memcached) — это простейшая модель, обеспечивающая максимальную скорость доступа по ключу. Они идеально подходят для кэширования, сессий и очередей сообщений.

Колоночные базы данных (Cassandra, HBase) оптимизированы для записи и чтения больших объёмов данных по определённым колонкам. Они широко используются в аналитических системах и приложениях с высокой нагрузкой. Графовые базы данных (Neo4j, ArangoDB) моделируют данные в виде узлов и рёбер, что делает их незаменимыми для социальных сетей, рекомендательных систем и задач анализа связей.

NoSQL-базы данных обычно не поддерживают полные транзакции ACID, однако многие современные решения предоставляют ограниченную поддержку транзакций на уровне отдельного документа или раздела.

Сравнение SQL и NoSQL

Выбор между SQL и NoSQL зависит от конкретных требований проекта. В следующей таблице приведено сравнение основных характеристик обоих подходов:

Критерий	SQL	NoSQL
Структура данных	Строгие таблицы	Гибкие документы
Масштабирование	Вертикальное	Горизонтальное
Схема	Фиксированная	Динамическая
Транзакции (ACID)	Полная поддержка	Ограниченная
Примеры	SQLite, PostgreSQL, MySQL	MongoDB, Redis, Cassandra
Использование	Финансы, учёт, CRM	Соцсети, IoT, логи

Таблица 1. Сравнение SQL и NoSQL подходов к хранению данных.

SQLite: работа с базой данных в Python

Подключение и создание таблиц

SQLite — это лёгкая встраиваемая реляционная база данных, которая не требует отдельного сервера. Все данные хранятся в одном файле, что делает SQLite идеальным выбором для небольших приложений, прототипов и обучения. В Python модуль sqlite3 входит в стандартную библиотеку, поэтому дополнительная установка не требуется.

Ниже приведён пример создания базы данных и таблицы:

```
import sqlite3

conn = sqlite3.connect("school.db")
cursor = conn.cursor()

cursor.execute("""
    CREATE TABLE IF NOT EXISTS students (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        age INTEGER,
        grade TEXT
    )
""")

conn.commit()
conn.close()
```

Функция sqlite3.connect() создаёт подключение к базе данных. Если файл school.db не существует, он будет создан автоматически. Объект cursor используется для выполнения SQL-запросов. Метод conn.commit() сохраняет изменения, а conn.close() закрывает подключение.

Основные операции CRUD

CRUD — это аббревиатура от Create (создать), Read (прочитать), Update (обновить), Delete (удалить). Эти четыре операции составляют основу работы с любыми базами данных.

INSERT — добавление записей:

```
import sqlite3

conn = sqlite3.connect("school.db")
cursor = conn.cursor()

cursor.execute(
    "INSERT INTO students (name, age, grade) VALUES (?, ?, ?)",
    ("Анна Иванова", 20, "A")
)

# Добавление нескольких записей
students = [
    ("Иван Петров", 21, "B"),
```

```
    ("Мария Сидорова", 19, "A"),
    ("Алексей Козлов", 22, "C")
]
cursor.executemany(
    "INSERT INTO students (name, age, grade) VALUES (?, ?, ?)",
    students
)

conn.commit()
conn.close()
```

Вместо прямой подстановки значений в строку запроса используются параметры (знак вопроса ?). Это защищает от SQL-инъекций — одного из самых распространённых типов атак на базы данных.

SELECT — чтение данных:

```
import sqlite3

conn = sqlite3.connect("school.db")
cursor = conn.cursor()

# Выборка всех записей
cursor.execute("SELECT * FROM students")
rows = cursor.fetchall()
for row in rows:
    print(row)

# Выборка отдельных столбцов
cursor.execute("SELECT name, grade FROM students WHERE age > 19")
results = cursor.fetchall()
conn.close()
```

UPDATE — обновление записей:

```
import sqlite3

conn = sqlite3.connect("school.db")
cursor = conn.cursor()

cursor.execute(
    "UPDATE students SET grade = ? WHERE name = ?",
    ("A+", "Анна Иванова")
)

conn.commit()
conn.close()
```

DELETE — удаление записей:

```
import sqlite3

conn = sqlite3.connect("school.db")
cursor = conn.cursor()
```

```
cursor.execute(
    "DELETE FROM students WHERE id = ?"
    (4,)
)

conn.commit()
conn.close()
```

Выборка данных с фильтрацией

SQL предоставляет мощные средства для фильтрации и сортировки данных. Оператор WHERE задаёт условия выборки, ORDER BY сортирует результаты, LIMIT ограничивает количество возвращаемых записей, а оператор LIKE позволяет искать по шаблону.

```
import sqlite3

conn = sqlite3.connect("school.db")
cursor = conn.cursor()

# Фильтрация по условию
cursor.execute("SELECT * FROM students WHERE grade = ? ORDER BY name", ("A",))

# Поиск по шаблону
cursor.execute("SELECT * FROM students WHERE name LIKE ? ", ("%Иван%",))

# Ограничение количества записей
cursor.execute("SELECT * FROM students ORDER BY age DESC LIMIT 3")

# Использование параметризованных запросов
min_age = 20
cursor.execute("SELECT name, age FROM students WHERE age ≥ ?", (min_age,))

results = cursor.fetchall()
for name, age in results:
    print(f"Студент: {name}, Возраст: {age}")

conn.close()
```

Параметризованные запросы (с использованием знака вопроса ?) — это критически важная техника безопасности. Никогда не подставляйте пользовательский ввод напрямую в SQL-запрос, поскольку это открывает уязвимость для SQL-инъекций, при которых злоумышленник может выполнить произвольный код в вашей базе данных.

PostgreSQL: продвинутая работа

Установка psycopg2

PostgreSQL — это мощная объектно-реляционная система управления базами данных с открытым исходным кодом. В отличие от SQLite, PostgreSQL работает в режиме клиент-сервер

и поддерживает расширенные возможности: полные транзакции ACID, сложные типы данных (массивы, JSON, геометрические типы), полнотекстовый поиск, наследование таблиц и многое другое.

Для работы с PostgreSQL из Python используется библиотека psycopg2. Установка выполняется командой:

```
pip install psycopg2-binary
```

Пакет psycopg2-binary включает предварительно скомпилированные библиотеки и не требует системных зависимостей, что делает установку быстрой и простой. Для production-окружений рекомендуется использовать psycopg2 с компиляцией из исходного кода.

Пример подключения и запросов

Ниже приведён полный пример работы с PostgreSQL:

```
import psycopg2
from psycopg2.extras import RealDictCursor

# Подключение к базе данных
conn = psycopg2.connect(
    host="localhost",
    database="mydb",
    user="postgres",
    password="password"
)
cursor = conn.cursor()

# Создание таблицы
cursor.execute("""
    CREATE TABLE IF NOT EXISTS products (
        id SERIAL PRIMARY KEY,
        name VARCHAR(100) NOT NULL,
        price DECIMAL(10, 2),
        stock INTEGER DEFAULT 0
    )
""")

# Вставка данных
cursor.execute(
    "INSERT INTO products (name, price, stock) VALUES (%s, %s, %s)",
    ("Ноутбук", 54999.99, 15)
)

# Чтение данных
cursor.execute("SELECT * FROM products WHERE price > %s", (10000,))
products = cursor.fetchall()
for product in products:
    print(product)

conn.commit()
cursor.close()
conn.close()
```

Обратите внимание на использование %s вместо ? для параметризованных запросов — это синтаксис, принятый в PostgreSQL. Для боевых приложений рекомендуется использовать пулы соединений (connection pooling) с помощью библиотеки psycopg2.pool или SQLAlchemy Engine, чтобы избежать накладных расходов на создание нового подключения для каждого запроса.

MongoDB: работа с NoSQL в Python

Установка PyMongo

MongoDB — это документоориентированная NoSQL-база данных, хранящая данные в виде BSON-документов (двоичный JSON). Каждая запись представляет собой документ с гибкой структурой: разные документы в одной коллекции могут содержать разные поля. Это делает MongoDB отличным выбором для приложений с быстро меняющейся схемой данных.

Для работы с MongoDB из Python используется библиотека PyMongo. Установка:

```
pip install pymongo
```

Для начала работы необходимо установить сам сервер MongoDB или использовать облачный сервис MongoDB Atlas, который предоставляет бесплатный тарифный план.

Основные операции

Рассмотрим основные операции с MongoDB на примере коллекции пользователей:

```
from pymongo import MongoClient

# Подключение к серверу
client = MongoClient("mongodb://localhost:27017/")

# Выбор базы данных и коллекции
db = client["myapp"]
users = db["users"]

# Вставка одного документа
user = {
    "name": "Елена Смирнова",
    "email": "elena@example.com",
    "age": 25,
    "tags": ["python", "django"]
}
users.insert_one(user)

# Вставка нескольких документов
new_users = [
    {"name": "Дмитрий Волков", "email": "dmitry@example.com", "age": 30},
    {"name": "Ольга Новикова", "email": "olga@example.com", "age": 27}
]
users.insert_many(new_users)

# Поиск одного документа
```

```
found = users.find_one({"name": "Елена Смирнова"})
print(found)

# Поиск с фильтрацией
results = users.find({"age": {"$gte": 25}})
for doc in results:
    print(doc["name"], doc["email"])

# Обновление документа
users.update_one(
    {"name": "Елена Смирнова"},
    {"$set": {"age": 26, "tags": ["python", "flask", "mongodb"]}}
)

# Удаление документа
users.delete_one({"name": "Дмитрий Волков"})
```

MongoDB использует богатый язык запросов с операторами: $eq (равно), $ne (не равно), $gt / $gte (больше / больше или равно), $lt / $lte (меньше / меньше или равно), $in (входит в список), $regex (регулярное выражение) и множеством других. Оператор $set используется для обновления отдельных полей документа, не затрагивая остальные.

В отличие от SQL-баз данных, MongoDB автоматически создаёт коллекции и индексы при первой записи, что значительно упрощает начальную разработку. Однако для production-окружения необходимо явно создавать индексы для обеспечения производительности запросов.

ORM в Python: SQLAlchemy

Что такое ORM

ORM (Object-Relational Mapping, объектно-реляционное отображение) — это технология, которая позволяет работать с базой данных с помощью объектно-ориентированных конструкций вместо написания SQL-запросов вручную. С помощью ORM каждая таблица базы данных представляется в виде Python-класса, каждая запись — в виде экземпляра класса, а каждый столбец — в виде атрибута этого класса.

SQLAlchemy — это самая популярная библиотека ORM для Python, которая предоставляет как высокоуровневый ORM-интерфейс, так и низкоуровневый доступ к SQL (Core). Основные преимущества использования ORM: отсутствие необходимости писать SQL-запросы вручную (хотя это возможно), автоматическая генерация типов данных на основе Python-классов, удобное управление миграциями схемы данных, переносимость между различными СУБД (SQLite, PostgreSQL, MySQL) без изменения кода.

ORM значительно упрощает сопровождение кода. При изменении структуры таблицы достаточно изменить соответствующий Python-класс, а ORM автоматически адаптирует SQL-запросы. Однако ORM имеет и недостатки: может быть медленнее, чем ручные SQL-запросы, сложнее для изучения и иногда генерирует неоптимальные запросы. Поэтому для критически важных участков кода SQLAlchemy позволяет использовать сырой SQL.

Пример использования SQLAlchemy

Ниже приведён полный пример определения моделей и выполнения CRUD-операций с помощью SQLAlchemy:

```
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.orm import declarative_base, sessionmaker

# Настройка подключения к SQLite
engine = create_engine("sqlite:///library.db")
Base = declarative_base()

# Определение модели
class Book(Base):
    __tablename__ = "books"

    id = Column(Integer, primary_key=True)
    title = Column(String(200), nullable=False)
    author = Column(String(100), nullable=False)
    year = Column(Integer)

# Создание таблиц
Base.metadata.create_all(engine)

# Создание сессии
Session = sessionmaker(bind=engine)
session = Session()

# Создание (Create)
book1 = Book(title="Python Crash Course",
             author="Eric Matthes", year=2019)
book2 = Book(title="Fluent Python",
             author="Luciano Ramalho", year=2015)
session.add_all([book1, book2])
session.commit()

# Чтение (Read)
books = session.query(Book).filter(Book.year ≥ 2015).all()
for b in books:
    print(f"{b.title} — {b.author} ({b.year})")

# Обновление (Update)
book = session.query(Book).filter_by(title="Fluent Python").first()
if book:
    book.year = 2022
    session.commit()

# Удаление (Delete)
session.query(Book).filter(Book.id == 1).delete()
session.commit()

session.close()
```

SQLAlchemy автоматически генерирует SQL-запросы на основе вызовов методов Python. Сессия отслеживает все изменения объектов и при вызове commit() отправляет соответствующие SQL-запросы в базу данных. Для управления изменениями схемы в production-окружениях используется библиотека Alembic, которая поддерживает миграции — версионирование структуры базы данных.

Практические рекомендации

Когда выбирать SQL

Реляционные базы данных (SQL) являются оптимальным выбором в следующих случаях:

Когда данные имеют чёткую, фиксированную структуру и могут быть организованы в таблицы. Финансовые системы, бухгалтерия, учётные приложения — все они требуют строгой типизации и целостности данных, которые гарантируют внешние ключи и ограничения. SQL-базы данных также незаменимы, когда приложению необходимы сложные аналитические запросы с объединением данных из нескольких таблиц, агрегацией и группировкой. Если проект предполагает множество связанных сущностей с жёстко определёнными отношениями (например, система управления заказами с клиентами, товарами и счетами), реляционная модель обеспечит целостность и согласованность данных на всех уровнях.

Когда выбирать NoSQL

NoSQL-базы данных лучше подходят для следующих сценариев:

Когда схема данных часто меняется или заранее неизвестна. Например, в системе сбора логов каждая запись может содержать разный набор полей. NoSQL отлично справляется с большими объёмами данных (big data), где требуется горизонтальное масштабирование на десятки или сотни серверов. Реальные приложения (real-time analytics, чаты, системы мониторинга) нуждаются в высокой скорости записи и чтения, которую обеспечивают key-value хранилища и документные базы данных. Иерархические и вложенные структуры данных (комментарии, категории, конфигурации) естественно представляются в виде документов, а не плоских таблиц.

Лучшие практики

Независимо от выбранной базы данных, соблюдение следующих рекомендаций поможет создать надёжное и эффективное приложение:

1. Всегда используйте параметризованные запросы — никогда не подставляйте пользовательский ввод напрямую в SQL-запрос. Это защищает от SQL-инъекций.
2. Закрывайте подключения к базе данных после использования или используйте контекстные менеджеры (with) для автоматического управления ресурсами.
3. Используйте пулы подключений (connection pooling) в production-окружении для уменьшения накладных расходов на создание соединений.
4. Добавляйте индексы на колонки, по которым выполняется частый поиск и фильтрация. Это может ускорить запросы в сотни раз.

5. Регулярно создавайте резервные копии базы данных и проверяйте процедуру восстановления. Используйте инструменты наподобие pg_dump для PostgreSQL или mongodump для MongoDB.
6. Используйте ORM (SQLAlchemy, Django ORM) для крупных проектов — это ускорит разработку, повысит читаемость кода и облегчит миграции.
7. Проводите нагрузочное тестирование и мониторинг производительности запросов, чтобы выявлять узкие места до того, как они станут проблемой.

Следуя этим принципам, вы сможете создавать надёжные, масштабируемые и безопасные приложения, которые эффективно работают с данными любого объёма и сложности.